TECHNICAL REPORT

MoodMirror: Lightweight Facial Expression Recognition with Convolutional Neural Networks for FER2013

EE4016 Group Project

City University of Hong Kong

Group 10

Final Technical Report

Due Date: April 25, 2026

Draft Date: April 24, 2026

Authors

KAPYA Zachariah Muya (Leader) [58494409]

WANG Xinan [58521809]

Cheng Wing Yan [58532928]

YIP Chung Hon [58737067]

Sit Yan Tung [57850588]

Revision History

Version	Date	Comments
0.1	2026-04-12	Integration snapshot prepared on presentation branch
0.2	2026-04-24	Verified rerun of saved model artifacts on current dataset split
1.0	2026-04-24	Publication-ready narrative draft with appendix and formal structure

Contents

Abstract

This report presents MoodMirror, a lightweight facial expression recognition system that classifies seven emotions from FER2013 grayscale facial images. The design goal is to provide a practical middle ground between two extremes: very large models that can perform strongly on FER benchmarks but are computationally expensive, and very small lightweight models that are efficient but often suffer noticeable performance degradation. In particular, this project targets realistic household and ordinary PC deployment, so model quality is evaluated together with CPU inference practicality rather than accuracy alone.

To realize this objective, we compare three principal paths: HOG plus SVM as the classical baseline, SimpleCNN as the deep learning baseline, and LiteCNN as the proposed lightweight architecture, with a CLCM-style model used as the replication comparison baseline. The integrated pipeline includes a unified data loader, reproducible training and evaluation scripts, model artifact loading, and a Streamlit-based demonstration interface supporting image upload, webcam snapshot, and live video inference with intelligent sampling.

On currently verified repository artifacts, test accuracy results are 43.23% for HOG plus SVM, 60.39% for CLCM, and 65.88% for LiteCNN. LiteCNN remains within the project parameter budget at 900,903 trainable parameters and outperforms CLCM by 5.49 percentage points on test accuracy in the present checkpoint set. This aligns with the proposal direction: exceed CLCM performance while staying under the predefined lightweight parameter threshold so deployment remains feasible on an ordinary CPU-based system.

The report documents achieved outcomes, unresolved gaps, and final submission readiness. Core goals for integration and comparative benchmarking are met. Remaining optimization work is centered on freezing a reproducible LiteCNN checkpoint at or above the 70% target and completing final standardized latency and ablation tables under controlled hardware conditions.

1 Introduction

1.1 Motivation

Facial expression recognition remains a challenging visual classification problem due to intra-class variation, inter-class ambiguity, illumination changes, and occlusions. In education and practical deployment contexts, model design must balance predictive performance against computational efficiency. The project therefore targets lightweight architectures suitable for CPU-constrained use while preserving competitive recognition quality.

1.2 Problem Statement

Given an input face image, predict one of seven emotion classes:

- angry
- disgust
- fear
- happy
- sad
- surprise
- neutral

1.3 Project Objectives

From the approved proposal, objectives are:

- O1: Build a complete FER pipeline (preprocessing, training, evaluation, and demo)
- O2: Establish HOG plus SVM and SimpleCNN baselines
- O3: Build LiteCNN with less than 1.5M parameters, less than 10 ms CPU inference target, and at least 70% accuracy target
- O4: Conduct ablation-guided optimization analysis
- O5: Deliver an interactive demonstration interface with intelligent webcam sampling
- O6: Beat the CLCM lightweight baseline

1.4 Contributions of This Report

This document contributes:

- A publication-style narrative aligned to technical report conventions
- Verified benchmark results from current saved artifacts
- Explicit objective mapping from planned targets to achieved outcomes
- A complete Appendix A for individual contribution reporting

2 Report Conventions and Notation

2.1 General Symbols

Symbol	Explanation
N	Number of samples
C	Number of classes ($C = 7$)
x	Input image tensor
y	Ground-truth class label
$\hat{y}$	Predicted class label
θ	Trainable model parameters
L	Training loss
$p(y = c \mid x)$	Predicted probability of class c
Acc	Classification accuracy
$F1_{macro}$	Macro-averaged F1 score

2.2 Dataset and Split Symbols

Symbol	Explanation
D_{train}	Training split data
D_{val}	Validation split data
D_{test}	Test split data
E	Epoch count
B	Batch size

2.3 Function Notation

Function	Meaning
$f_\theta(x)$	Model forward pass producing logits
$softmax(z)$	Probability normalization of logits
$argmax(p)$	Predicted class index from probability vector
$M(\theta, D)$	Evaluation of model M with parameters θ on dataset D

3 Project Organization and Integration Evidence

3.1 Branch Consolidation

The final working branch is presentation, consolidating source contributions from:

- annie_m2_code
- jasmine_code
- master
- muyas_code

Repository branch inventory additionally includes remote branches for M4 and M5 paths, while integrated root-level evidence is captured in the merged branch.

3.2 Merge Governance

A conflict-safe manifest was used to preserve reproducibility:

- Unique paths promoted to root
- Conflicting files explicitly tracked and resolved

Conflicting file categories included repository configuration and core model dependency definitions.

3.3 Integrated Artifact Set

The consolidated implementation includes:

- Unified model definitions and training scripts
- Shared FER2013 folder-aware data loader
- HOG artifact training and loading path
- Demo application with model-agnostic adapter registry
- Frozen model artifacts for benchmark and demo use

4 Data, Preprocessing, and Experimental Protocol

4.1 Dataset

The project uses FER2013 in folder form:

- data/fer2013/train by class
- data/fer2013/test by class

The current test set used in verified evaluation contains 7,178 samples.

4.2 Preprocessing Pipeline

Training-time augmentation includes:

- random horizontal flipping
- bounded rotation
- affine translation
- mild brightness and contrast jitter

Inference-time preprocessing includes:

- grayscale normalization
- resize to 48 by 48
- tensor conversion and normalization with mean 0.5 and std 0.5

4.3 Evaluation Metrics

Primary metrics:

- test accuracy
- macro F1

Secondary deployment metrics:

- trainable parameter count
- average inference latency

4.4 Evidence Classification

To maintain publication-level integrity, results are reported as:

- Verified current results: direct rerun on present saved artifacts
- Historical reported values: documented from earlier runs and notes

Only verified current results are used for final claim tables in this draft.

5 Methods

5.1 HOG plus SVM Baseline

The classical baseline extracts HOG descriptors and trains a linear SVM classifier. This baseline anchors performance against non-deep-learning approaches and supports confusion matrix analysis and macro F1 reporting.

5.2 SimpleCNN Baseline

SimpleCNN is a four-block convolutional architecture with batch normalization, pooling, and dropout regularization, followed by global average pooling and a linear classifier.

Trainable parameters: 390,599.

5.3 LiteCNN Proposed Model

LiteCNN uses residual depthwise-separable blocks for computational efficiency while retaining representational depth. The architecture progressively expands channels and uses a lightweight classifier head.

Trainable parameters: 900,903.

5.4 CLCM Replication Baseline

The CLCM path provides a lightweight replication baseline for objective O6 comparison.

Trainable parameters: 117,383.

6 System Integration and Demo Architecture

6.1 Unified Adapter Contract

The application integrates all model families through a single prediction contract containing:

- class index
- class label
- confidence
- class probability vector
- inference latency
- model identity

This decouples frontend behavior from model internals and significantly reduces integration risk.

6.2 Input Modes

The deployed demo supports:

- image upload inference
- webcam snapshot inference
- live video inference via streamlit-webrtc

6.3 Intelligent Sampling for Live Inference

For webcam and live video, the system supports:

- timed inference mode
- change-triggered mode using L2 frame difference

Runtime counters expose practical deployment behavior:

- total frames

- inference calls
- skipped frames
- estimated inference reduction percentage

7 Results

7.1 Verified Current Benchmark Table

All values below were re-evaluated on April 24, 2026 using current model artifacts under the integrated environment.

Model	Test Accuracy (%)	Macro F1	Parameters	Avg Latency (ms/sample)	Evidence Status
HOG plus SVM artifact	43.23	0.3803	N/A	N/A	Verified current
SimpleCNN checkpoint	25.65	0.0840	390,599	1.05	Verified current
LiteCNN checkpoint	65.88	0.6340	900,903	9.48	Verified current
CLCM checkpoint	60.39	0.5268	117,383	2.15	Verified current

7.2 Comparative Interpretation

- LiteCNN is the strongest current checkpoint and exceeds CLCM by 5.49 percentage points on test accuracy.
- LiteCNN remains under the 1.5M parameter budget.
- Current SimpleCNN checkpoint performance is substantially below historical notes, indicating checkpoint lineage or compatibility drift that should be reconciled before final archival claims.

7.3 Historical Context (Non-claim Support)

Project notes and earlier run summaries include historical values near:

- SimpleCNN: about 63% test
- LiteCNN: about 66% test

These values are useful context but are not treated as final claims unless reproduced from the frozen final artifact set.

8 Objective Achievement Matrix

Objective	Requirement	Status	Evidence-Based Comment
O1	End-to-end pipeline	Achieved	Data loader, training, evaluation, and demo are all integrated
O2	Baseline establishment	Achieved	HOG plus SVM and SimpleCNN paths implemented and runnable
O3	LiteCNN target package	Partially achieved	Parameter budget met; current accuracy below 70%; latency near target in measured run
O4	Ablation quantification	Partial	Ablation-ready scripts and configs exist; final controlled table still needed
O5	Interactive demo with smart sampling	Achieved	Upload, webcam, and live video with sampling controls are implemented
O6	Beat CLCM	Achieved	LiteCNN current test accuracy exceeds CLCM current test accuracy

9 Discussion

9.1 What Was Achieved Well

The strongest success is engineering integration quality. The system demonstrates consistent data handling, unified adapter design, artifact-backed model loading, and deployment-

oriented sampling in a single app workflow. This directly supports reproducibility and presentation reliability.

9.2 Remaining Scientific and Reporting Gaps

Three finalization gaps remain before report freeze:

- freeze a fully reproducible LiteCNN checkpoint at or above the 70% target
- reconcile SimpleCNN historical versus current artifact discrepancy
- finalize ablation and latency reporting under one controlled protocol

9.3 Threats to Validity

Potential threats include:

- checkpoint provenance mismatch
- hardware-dependent latency variation
- split or preprocessing drift across separate branch-era runs

These are manageable with final artifact freeze and documented rerun scripts.

10 Conclusion

MoodMirror delivers a complete lightweight FER pipeline from model development to integrated application deployment. In current verified evaluation, LiteCNN outperforms the CLCM baseline while satisfying parameter-budget constraints, and the demo system is fully operational across all required interaction modes.

The final optimization objective is to close the remaining accuracy gap to the 70% target under a frozen and reproducible final checkpoint set. With this final step and completed controlled ablation tables, the project is positioned for a strong final technical submission.

11 Compliance with Final Deliverable Requirements

Professor-required items and current status:

- Final technical report (minimum 25 pages): this draft provides full structured content for LaTeX expansion and formatting
- PowerPoint presentation: to be finalized with benchmark and architecture visuals
- Python source code: available in repository
- Demo video (3 to 4 minutes): app supports full demonstration flow
- Appendix A individual contributions: included below
- Single zip bundle for Canvas: pending final packaging

References

1. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.
2. Y. LeCun, Y. Bengio, and G. Hinton, Deep learning, *Nature*, 2015.
3. N. Dalal and B. Triggs, Histograms of oriented gradients for human detection, CVPR, 2005.
4. C. Cortes and V. Vapnik, Support-vector networks, *Machine Learning*, 1995.
5. A. Krizhevsky, I. Sutskever, and G. Hinton, ImageNet classification with deep convolutional neural networks, NIPS, 2012.
6. A. Howard et al., MobileNets: Efficient convolutional neural networks for mobile vision applications, 2017.
7. M. C. Gursesli et al., Facial Emotion Recognition Through Custom Lightweight CNN Model, *IEEE Access*, 2024, doi:10.1109/ACCESS.2024.3380847.
8. K. He et al., Deep residual learning for image recognition, CVPR, 2016.
9. F. Chollet, Xception: Deep learning with depthwise separable convolutions, CVPR, 2017.
10. D. P. Kingma and J. Ba, Adam: A method for stochastic optimization, ICLR, 2015.

Member	Responsibility Area	Completed Work	Outcomes
M1 (Leader)	Integration, demo, coordination	Branch consolidation, adapter-based demo, intelligent sampling integration, final benchmark reconciliation support	End-to-end runnable system and demonstration readiness
M2	Data and classical baseline	FER data loading pipeline, HOG plus SVM baseline workflow, baseline evaluation setup	Reliable shared data foundation and classical benchmark path
M3	CNN model development	SimpleCNN and LiteCNN architectures and training scripts	Lightweight deep-learning model backbone for project evaluation
M4	Experimental design and ablations	Configuration and training-parameter experimentation framework	Ablation-ready infrastructure and optimization control points
M5	Evaluation and analysis support	Metrics interpretation and reporting support for final synthesis	Improved result interpretation and report alignment

A Individual Contributions of the Group Project

A.1 Member Responsibilities and Outcomes

A.2 Integration-Level Team Achievement

- Shared contracts enabled independent development and late-stage merge stability.
- Artifact-based model loading enabled reproducible demo behavior.
- Unified output schema provided fair model comparison in the same interface.

A.3 Remaining Work Allocation Before Final Submission Lock

- Final optimization and rerun ownership: LiteCNN freeze candidate and latency table
- Reproducibility fix ownership: SimpleCNN checkpoint lineage verification
- Final report figure ownership: confusion matrices, architecture diagram, demo screenshots

B Recommended Figure and Table Insertion Plan

To align with publication-style layout in your final template, include:

- Figure B1: System architecture overview
- Figure B2: Demo interface snapshots for three input modes
- Figure B3: HOG plus SVM confusion matrix
- Figure B4: LiteCNN confusion matrix and error concentration highlights
- Table B1: Verified benchmark table
- Table B2: Objective achievement matrix
- Table B3: Final ablation summary
- Table B4: Latency benchmark under fixed hardware protocol

C Packaging Note for Final Submission

Before final zip upload to Canvas, include:

- final report PDF
- presentation slides
- source code snapshot
- demo video file
- appendix-complete report source and any supplementary plots